

K-Fold Cross-Validation for Model Validation and Comparison

1. What You'll Learn in This Section

In this lesson, you'll learn to:

- Explain why a single train/test split can give a misleading performance number, and why cross-validated comparisons need shared fold divisions.
- Describe how K-fold cross-validation rotates every data point through the test role.
- Interpret the mean and standard deviation of cross-validation scores to judge reliability.
- Apply K-fold cross-validation in Python to fairly compare two algorithms and build a final deployable model.

2. Detailed Explanation

Why one train/test split can mislead you

A single split of your data into training and test sets produces exactly one accuracy number — for example, 80% for a logistic regression model. The catch is that this number only reflects performance on whichever rows randomly ended up in the test set.

Picture a weather classification problem with three classes — cloudy, rain, and sunny — predicted from parameters like wind speed and cloud cover. Split the data 80% train / 20% test, and one of two things can happen purely by chance:

- The 20% test rows happen to be easy, obvious cases (clear rain days, clear sunny days). The model then scores unusually well, maybe 85% or even 95%, not because it generalizes well but because the test set was easy.
- The test rows happen to be genuinely hard, ambiguous cases. The same model, trained on the same data, might score only 60% on that split.

Both outcomes are artifacts of which rows landed in the test set, not a fair assessment of model quality. Relying on one split means your reported number might just reflect a lucky or unlucky draw.

Folds and the K-fold rotation process

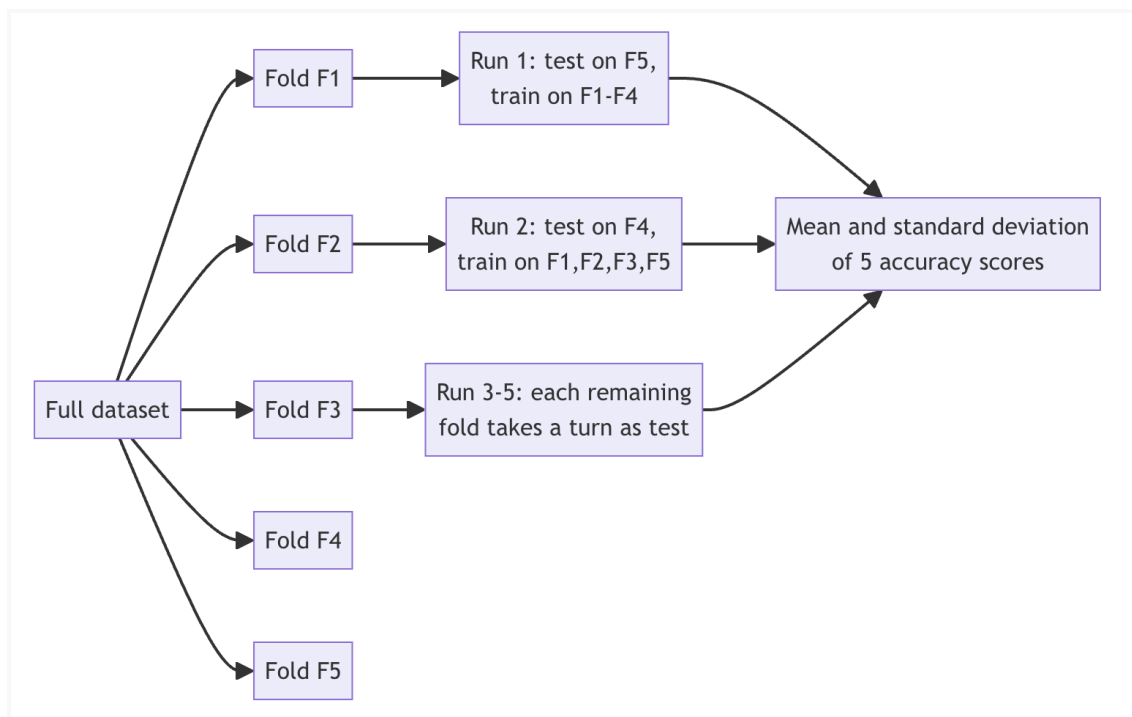
A fold is a mutually exclusive, non-overlapping subset of the full dataset — a plain-English way to say "one slice of the data that never overlaps with another slice." If a dataset of 100 points is divided with $K = 5$, you get five folds, each holding 20 data points.

K-fold cross-validation runs K separate training/testing experiments using the same algorithm. Each experiment holds out a different fold as the test set and trains on the remaining $K - 1$ folds. With five folds labeled F1 through F5:

Run	Train on	Test on
1	F1, F2, F3, F4	F5
2	F1, F2, F3, F5	F4
3	F1, F2, F4, F5	F3

4	F1, F3, F4, F5	F2
5	F2, F3, F4, F5	F1

Each run still keeps the overall 80/20 ratio, but which specific rows fall into train versus test changes every time. Across all K runs, every single data point gets used as a test point exactly once, and as a training point in the other K – 1 runs.



Reporting results: mean, standard deviation, and why shuffle matters

The final reported result is the mean of the K accuracy scores — one score per run. This mean is more reliable than any single split's number because it reflects performance across the whole dataset acting as test data at some point.

Alongside the mean, a standard deviation is computed across the K scores. This represents the spread — how much individual fold scores deviate from the mean in either direction. Cross-validation doesn't eliminate variability; it makes that variability visible instead of hiding it behind one lucky or unlucky split.

A useful analogy: imagine a class of 10 students whose mean math score is 80 out of 100.

- If the standard deviation is high — scores ranging from 40 or 50 up to 100 — the mean of 80 is misleading. Some students are doing poorly even though the average looks good.
- If the standard deviation is low — scores tightly clustered between 70 and 90 — the mean of 80 is trustworthy.

The same logic applies to cross-validation: a high mean with a low standard deviation is far more trustworthy than a high mean with a large standard deviation. The latter signals inconsistent performance depending on which data ends up tested.

Watch out for: before splitting data into folds, shuffle it (`shuffle=True`) so folds aren't built from data sitting in its original, possibly sorted, order. If a feature like income happens to be sorted in the raw data, an unshuffled split could put all the highest incomes in one fold and all the lowest in another, biasing that fold's results. It's the same reason you reshuffle a deck of cards between games — without reshuffling, whoever collects the winning hand keeps an unfair advantage next round purely from leftover ordering.

Manual worked example: five folds and a six-fold check

Using sets A, B, C, D, E as the five folds:

- Round 1: test on Set A, train on B, C, D, E

- Round 2: test on Set B, train on A, C, D, E
- Round 3: test on Set C, train on A, B, D, E
- Round 4: test on Set D, train on A, B, C, E
- Round 5: test on Set E, train on A, B, C, D

Five performance scores result, one per round, with none of the test portions overlapping. For a loan-approval classification scenario, one round of five-fold cross-validation produced these illustrative fold accuracies:

Fold	Accuracy
1	0.68
2	0.93
3	0.81
4	0.81
5	0.688

Mean accuracy: 0.78, standard deviation: 0.094 — considered an acceptable spread.

This 0.78 mean is more representative of the model's true performance than any individual fold's score, because it comes from the entire dataset having served as test data across all five runs.

In a separate six-fold run on the same kind of data, scores ranged from a worst case of about 65% to a best case of about 90% — a swing of roughly 25 percentage points.

Neither the 65% nor the 90% number is individually reportable. The mean across all

folds is what should be reported, because it represents a genuine average rather than an extreme case.

Case study: loan approval — features and why one split still fails

A financial bank wants a machine learning model to decide whether a loan application should be approved or not approved, replacing or supplementing manual review by a team of experts. The model uses four features:

- Income source / income amount — actual income in monetary terms. The modeled feature is the numeric income value (the reputation of the income source also matters conceptually, but the number is what's fed into the model).
- Credit score — reflects payment history: how many fixed deposits (FDs) were completed without being broken early, and how consistently credit card bills were paid in full and on time. Paying only the minimum due still damages the score, even though it avoids a late fee. Higher is better, up to roughly 800.
- Debt ratio — the proportion of a person's total assets acquired using loan financing. If 20% of someone's assets were bought on loan, their debt ratio is 0.2. Lower is better — banks are reluctant to extend a new loan to someone with a high debt ratio, since it signals higher risk of non-repayment.
- Employment years — how long the applicant has held their job. A moderate span (roughly 7 to 15 years) is favorable, suggesting stability and remaining working years. A very high number (e.g., 25) can turn slightly negative, since the applicant may be nearing retirement.

Even here, a single split can mislead: the test portion might, purely by chance, contain unusually high employment-years values associated with non-approval in the data pattern. A well-trained model can then appear to perform poorly, even though it

generally works well. The reverse can also happen — training data could accidentally be too "easy," leaving the model unable to handle harder, unseen combinations.

If a model is evaluated with $K = 10$ (ten-fold cross-validation), its performance is observed ten times across different train/test combinations, versus only once with a single split. Even if two people report the identical accuracy figure (say, both report 85%), a client is justified in trusting the number from ten-fold cross-validation more. It was validated across ten separate experiments rather than one arbitrary division.

Comparing two algorithms fairly

K-fold cross-validation can also compare two different algorithms — for example, logistic regression versus a decision tree classifier — rather than just evaluating one algorithm repeatedly. This kind of comparison is only meaningful if both algorithms are judged on identical conditions.

Critical rule: both algorithms must be evaluated on exactly the same fold divisions. It would be unfair to give logistic regression one set of train/test fold combinations while giving the decision tree a different set. The two algorithms must see identical rows in "fold 1," "fold 2," and so on. With a single split, this was automatically satisfied because both algorithms trained and tested on the identical split. With K-fold cross-validation, this discipline must be enforced deliberately: generate the folds once, then run every algorithm being compared against that identical set of fold combinations.

Once both algorithms are evaluated on the same folds, their means and standard deviations become directly comparable. The algorithm with the better mean — and an acceptable standard deviation — can be judged the stronger choice for the dataset.

Building the final deployable model

Running K-fold cross-validation on logistic regression yields K different trained models (one per fold), each with its own weight parameters (for example, W_0 and W_1 in a simple one-feature logistic function). None of these K individual models goes to the client. Instead, the final deployed model is built by averaging the weight parameters across all K folds.

For a simple one-feature logistic regression, the model form is:

$$F(x) = 1 / (1 + e^{-(W_0 + W_1 * x)})$$

The deployed version uses cross-validated weights:

$$W_0_CV = (W_0_fold1 + W_0_fold2 + \dots + W_0_fold5) / 5$$

$$W_1_CV = (W_1_fold1 + W_1_fold2 + \dots + W_1_fold5) / 5$$

$$F(x) = 1 / (1 + e^{-(W_0_CV + W_1_CV * x)})$$

This averaged function isn't the output of any single fold's run — it's constructed by averaging the weight parameters obtained from all K runs. This is the function delivered to the client for prediction on new data.

Python workflow: from data to cross-validated scores

The core imports for this workflow:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```


`KFold` creates K non-overlapping folds of the dataset. `cross_val_score` is a convenience function: given a model, the data (X, y), and a fold specification, it automatically trains and tests the model across all K fold combinations. It returns an array of K scores, internally handling "train on four folds, test on the last fold" for each rotation. `sklearn.metrics` also provides `f1_score`, `precision`, and `recall`; this workflow uses `accuracy_score`.

Synthetic loan dataset. Eighty synthetic applicants are generated with four features, each drawn from a uniform distribution via `np.random.uniform`:

Feature	Range
Income	20 to 120
Debt ratio	0 to 0.6
Credit score	500 to 820
Employment years	0 to 20

The target variable (`approved`) is generated using a linear combination of the features passed through a sigmoid function:

```
z = 0.03 * (credit_score - 650) - 6 * debt_ratio + 0.02 * income + 0.15 *  
employment_years - 1.5  
probability_of_approval = 1 / (1 + e^(-z))
```

Each term's sign shows its effect on approval likelihood:

- Credit score carries a positive coefficient (0.03, after centering by subtracting 650).
- Debt ratio carries a negative coefficient (-6).
- Income is positive (0.02).
- Employment years is mildly positive (0.15).
- A bias term of -1.5 shifts the whole function.

To decide the final binary label, 80 random values are drawn uniformly between 0 and 1. Each applicant's probability of approval is compared against their own random draw: if the probability exceeds the draw, `approved = 1`, otherwise `approved = 0`. This mirrors real-world lending, where two applicants with the same underlying score don't always get the same decision — other external factors can tip a borderline case either way. With 80 applicants generated this way, the approval rate (mean of `y`) came out to 0.34, meaning 34% of applicants were approved.

Baseline: repeated single splits. A function `single_split_accuracy(random_state)` splits `X` and `y` with `train_test_split(test_size=0.25)`, trains a `LogisticRegression(max_iter=2000)` model, and computes `accuracy_score` on the test split. Called six times with `random_state` values 0 through 5, it produced roughly 0.75, 0.90, 0.70, 0.85, 0.60, and 0.80 — a spread of about 0.25 to 0.30 between the lowest and highest split. This variability is driven entirely by which rows randomly fell into the test set. It shows that repeated single splits still don't give a controlled, reliable comparison, since there's no guaranteed rotation of every data point through the test role.

Controlled K-fold cross-validation fixes the fold structure explicitly:

```
kf = KFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(model, X, y, cv=kf)
```

`n_splits=5` creates five folds. `shuffle=True` randomizes row order before splitting, avoiding bias from any pre-existing sorted order. `random_state=42` fixes the shuffle so the same five folds are produced every time the notebook reruns. `cross_val_score` trains logistic regression (2,000 iterations per fit) five times, once per fold rotation, returning scores such as 0.688, 0.93, 0.81, 0.680, and approximately 0.81 — aligning with the illustrative example (mean ≈ 0.78).

Comparing two algorithms fairly with `cv_summary`. A helper function

`cv_summary(model, X, y, cv)` runs `cross_val_score` for a model and fold object, returning the mean score, the standard deviation, and the full array of per-fold scores.

Two models are defined:

```
models = {  
    "Logistic Regression": LogisticRegression(max_iter=2000),  
    "Decision Tree Classifier": DecisionTreeClassifier()  
}
```

A loop calls `cv_summary(model, X, y, kf)` for each model using the same `kf` object created earlier, ensuring both algorithms are evaluated on identical fold divisions. The results are combined into a pandas DataFrame, rounded to three decimals, and transposed for readability.

Logistic regression achieved a higher mean accuracy (approximately 0.78, close to 0.80) than the decision tree classifier (mean approximately 0.70). Logistic regression's standard deviation was slightly higher than the decision tree's but still within an acceptable range. This aligns with theory: logistic regression is a learning-based algorithm that fits weight parameters through training. A decision tree is rule-based, using if-else splits governed by a metric such as the Gini index — a comparatively less sophisticated mechanism for deriving decision boundaries. A learning-based algorithm is generally expected to outperform a rule-based one when sufficient data is available.

The final comparison DataFrame (mean and standard deviation per model) is plotted as a horizontal bar chart, with mean accuracy determining each bar's length. Standard deviation is passed as `xerr` (an error bar in the X direction), visually showing both the central estimate and its spread for each algorithm.

A few clarifications worth knowing

- F1 score versus accuracy is a metric choice independent of the cross-validation decision — you can report either accuracy or F1 score with K-fold cross-validation, just as you would with a single split. F1 score is the safer choice specifically when a dataset is imbalanced (many more data points for one class than another); on a roughly balanced dataset, accuracy and F1 tend to be similar.
- When a decision tree might beat logistic regression: theoretically, a learning-based algorithm should perform at least as well as a rule-based one. However, with very small datasets, learning-based algorithms may fail to estimate reliable weight parameters, and in that specific scenario a decision tree can outperform logistic regression. This is a general property of dataset size versus algorithm type.
- What actually gets delivered to a client: rather than handing over raw weight parameters, libraries like scikit-learn package the trained model as a complete callable function or object. It's typically saved and delivered as a model file (for example, a `.model` file). Alternatively, it can be a notebook with instructions specifying which cell to edit with new test data and which cell to run to see predictions.

3. Key Takeaways

- A single train/test split reports one accuracy number that can be misleadingly high or low purely because of which rows landed in the test set.

- K-fold cross-validation rotates every fold through the test role exactly once, producing K scores instead of one.
- The mean of the K scores is the reliable headline result; the standard deviation tells you how consistent that result is — low spread alongside a high mean is the trustworthy combination.
- Always shuffle the data (`shuffle=True`) before creating folds to avoid bias from pre-existing sorted order.
- When comparing two algorithms, evaluate both on the exact same fold divisions (the same `kf` object) — otherwise the comparison isn't fair.
- The final deployable model is built by averaging each fold's weight parameters, not by picking one fold's model.

Mental model: Think of K-fold cross-validation as giving every slice of your data a fair turn in the spotlight as the "test." The model is then judged by its average performance across all those turns, instead of trusting the verdict of just one spotlight moment.